

Redlined and Left Behind: Housing Unaffordability and Displacement Risk in Charlotte, NC

DTSC 2302

Professors Schuckers and Ageenko

Ayia Boutanassa, Brady Dunn, Bryce Jackson, Eric Testen, Garrison High

May 4, 2026

Disclaimer on the use of AI: This project made use of a combination of large language models such as Claude (Anthropic) and ChatGPT as an aid for the following purposes: debugging and code troubleshooting, visualization enhancement, refining models with suggested hyper parameters, handling class imbalance, and drafting markdown summaries of the process in each notebook. All analytical decisions, including model selection, feature choices, data sources, and conclusions were made by the project team. AI-generated content was reviewed, edited, and validated by team members before inclusion. The models, code, and findings presented in this report are the work of the authors.

Introduction

As young adults, our focus naturally starts shifting towards becoming independent individuals, including the goal of living on our own and eventually becoming homeowners to achieve part of the American Dream. However, after experiencing high rental costs firsthand, this goal starts to feel somewhat impossible. Housing affordability seems to be an increasingly urgent issue, especially in Charlotte, North Carolina. This sparked our interest in identifying the key factors that most contribute to housing unaffordability and then use that to identify the neighborhoods most at risk of displacement. The topic is even more so interesting due to the rising housing costs disproportionately affecting lower-income families and historically marginalized communities. The increase in housing cost burden frequently forces residents to move out of their homes and reinforces the legacy of discriminatory practices such as redlining. From 1935 until 1968, redlining was used to mark “hazardous” neighborhoods in terms of investment which was usually related to neighborhoods with a high population of minority groups. This raises question of whether old policies continue to frame present-day outcomes.

For our investigation, we drew on four data sources: neighborhood socioeconomic and housing characteristics from the city of Charlotte’s Quality of Life Explorer, displacement vulnerability from Charlotte’s Open Data Portal, neighborhood boundary definitions from the city’s ArcGIS REST API, and historical Homeowner’s Loan Corporation redlining zone data from the University of Richmond’s “Mapping Inequality” project. We merged these sources through a SQLite database across 458 Neighborhood Profile Areas (NPAs). Three features were engineered from the raw data: `housing_cost_burden` which is a normalized composite index of rental and ownership affordability relative to a neighborhood’s median income;

displacement_risk is a binary encoding of the city's vulnerability classification; and crime_total, which is a composite of property crimes and violent crime rates.

When curiosity sparked around this topic, brainstorming different classification and regression models took place. After much refining and model training, our team ended up with an XGBoost model for the classification of neighborhoods at risk of displacement and a Generalized Additive Model (GAM) of four features as our best performing model for the regression model. Throughout this report, we will break down the journey of deciding on the selection of these two models and how we managed to intertwine both models into reaching our analysis of the lasting impact of historical redlining on present day housing cost burden and displacement risk.

Regression Model

When examining housing unaffordability across Charlotte, a regression framework was used to model housing cost burden as a continuous outcome. The modeling process shows a structured pipeline, beginning with data loading from a database of 458 observations and 23 features, followed by custom train/test splits. This then led to model refinement. The goal of this analysis was not only to achieve strong predictive performance, but also to better understand how different neighborhood characteristics contribute to variations in housing burden.

Multiple Linear Regression

The process began with a Multiple Linear Regression model using Ordinary Least Squares (OLS) to establish a baseline. A wide range of features was included initially, covering socioeconomic conditions, housing characteristics, demographic factors, and measures of accessibility. Prior to modeling, the features with high multicollinearity were removed during the explanatory data analysis (EDA). To refine the model, Recursive Feature Elimination (RFE) was used to identify the most relevant predictors using cross-validation and variables exhibiting high multicollinearity were removed to improve stability. The RFE results offered that model performance, using adjusted R^2 , was best around $k = 4$ features. These features specifically being household_income, home_ownership, food_nutrition, and age_of_residents.

Although the optimal subset of predictors contained four features, a 7-feature model was initially chosen to capture the broader neighborhood dynamics. These features included new_constructions, transit_proximity, and grocery_proximity. This decision was supported by having nearly identical adjusted R^2 values of 0.719 versus 0.718. This suggests minimal performance reduction while improving interpretability.

The OLS model was fitted using statsmodels and its performance was evaluated using R^2 and RMSE on both training and test sets. With this, the model achieved a test R^2 of approximately 0.69 with an RMSE near 0.089 and a train R^2 of about 0.73 with an RMSE of 0.0854. While these results showed moderate predictive power, the statistical results showed that both transit_proximity and grocery_proximity were not significant predictors as their p-values were greater than 0.05, suggesting that these variables added noise rather than any meaningful insight to the explanatory value. (See Appendix B, Table B1)

Diagnostics

Taking it a step further to assess model validity, the residual diagnostics were generated in coding, including different plots such as histograms, Q-Q plots, and residuals vs fitted plots. These outputs revealed two key violations of linear regression assumptions; First non-linearity, showing U-shaped residual pattern with LOWESS smoothing and second, heteroscedasticity, a fan-shaped spread of residuals. Additionally, five extreme outliers were identified when using a 3-standard-deviation threshold. Investigation of these observations showed that they correspond to neighborhoods with unusually high housing burden, which was relative to income, confirming they are real phenomena rather than error in data. (See Appendix B, Figures B1–B2)

Polynomial Regression Models

Given these limitations, alternative modeling strategies were explored. Polynomial regression models, with degrees 2-4, were considered to introduce non-linearity. These models implemented the usage of a pipeline (PolynomialFeatures and LinearRegression). The model comparison showed when using degree 2, an R^2 of approximately 0.83 and an RMSE of around 0.067 were achieved and when using degree 3 and 4, severe overfitting and poor generalization

occurred. Although the degree 2 model significantly improved the predictive performance, its interpretation was limited due to complex feature interactions. (See Appendix B, Figures B3–B4)

Generalized Additive Model

A more flexible approach was then implemented using a Generalized Additive Model (GAM) using spline features for each predictor, which allowed to balance flexibility and interpretability. The GAM was trained using grid search. This was to optimize the smoothing parameters. With the 7-feature set the model achieved an R^2 of about 0.83 and an RMSE of around 0.0668. The diagnostic plots generated in the code delivered clear improvements over OLS, which involved more symmetric residual distribution and reduced heteroscedasticity. (See Appendix B, Table B2, Figure B5)

Final Model Selection

More analysis using partial dependencies revealed that the `new_constructions`, `transit_proximity`, and `grocery_proximity` all contributed minimal explanatory power. These features were removed and the final model was retrained on the four key predictors identified earlier. The final 4-feature GAM produced the strongest results of a test R^2 of ~ 0.83 and a test RMSE of about 0.66. To evaluate generalization, a 5-fold cross validation was performed on the full dataset which then resulted in a mean R^2 of 0.8059 ± 0.0815 and a mean RMSE of 0.0713 ± 0.0181 . These results were consistent with test performance. This suggests that the model generalizes well despite some variability due to extreme outliers. (See Appendix B, Table B3, Table B4)

The partial dependence plot offered much insight to the drivers of the housing cost burden experienced by these neighborhoods. Household income has a very steep, negative curve that flattened the mean. This shows neighborhoods that have a below average income experience higher cost burden in housing. Home ownership has a negative relationship, a higher rate in home ownership confirms that renters are more cost burdened. Age of residents shows a lower burden in younger neighborhoods while older populations experience rising burden. This is consistent with tenured residents who rely on fixed incomes such as retirement or Social Security to absorb appreciating costs in housing. Finally, food nutrition is our proxy variable to poverty rates. The partial dependency shows it being flat at low to moderate levels, then spikes suddenly at a high reliance once it reaches a threshold of 4 on the scaled values. This could be used during ongoing monitoring to indicate when a neighborhood is approaching a cost burden crisis. (See Appendix B, Figure B6)

Overall, the modeling process demonstrated progression from a conventional multilinear approach to a much more flexible, accurate, non-linear model. The final GAM not only improves predictive performance but also captures complex, non-linear relationships between neighborhood characteristics and housing cost burden. This made it the most appropriate model for this analysis.

Classification Model

To predict displacement risk, multiple classification models, such as, Logistic Regression, Random Forest, K-Nearest Neighbors (KNN), and XGBoost models, were developed and evaluated. The main approach focused on balancing the model's predictive performance along with its' ability to accurately classify high-risk cases, with crucial emphasis on recall due to the importance of minimizing the amount of missed displacement cases.

Model Development Approach

The modeling process began with Logistic Regression as a baseline model. The initial model uses all the available features, excluding the target variable, the ID variable, and housing_cost_burden to avoid redundancy and performance inflation. To address class imbalance, SMOTE and class weights were applied. Cross-validation results showed strong performance of a mean F1 score of about 0.73 and a recall score of around 0.84. Different thresholds were tested out, but threshold tuning showed that a lower decision threshold, specifically at 0.3, significantly improved recall at about 0.90, which made it more fitting for classification of high-risk cases. (See Appendix C, Figure C1)

Two more Logistic Regression models were made using reduced feature sets. The first reduced model had 5 manually chosen features. This resulted in higher recall of about 0.93, but a significantly lower precision of 0.52, which suggested too many false positives. The second model was based on the top 5 features on the feature importance list. This second reduced model resulted in an F1 score of about 0.76 and a recall of around 0.90. This shows that a carefully selected subset of features can retain predictive power meanwhile improving interpretability. (See Appendix C, Figures C2–C3)

The next model implemented was a Random Forest model with the goal of capturing any non-linear relationships within the dataset. This model achieved a mean cross-validation F1 score of around 0.73 and an improved test performance of F1 at about 0.81 and recall at around 0.86. Compared to the Logistic Regression model, the Random Forest provided better balance between precision and recall and reduced misclassification rates. This indicated that non-linear interactions are important in predicting displacement risk. (See Appendix C, Figure C4)

The next step was testing out several K-Nearest Neighbors (KNN) models. They were tested with different feature selection strategies. While performance improved with feature selection, the best F1 being at about 0.75 and recall at approximately 0.83, the KNN models were generally more sensitive to noise and high dimensionality. Although they were considered competitive models, they did not outperform Random Forest nor XGBoost models in the balance and consistency. (See Appendix C, Figures C5–C8)

Finally, an XGBoost model was created and then tuned using grid search. We addressed the class imbalance using the `scale_pos_weight` parameter. The best model achieved a cross-validation F1 score of 0.75. On the test set, where a threshold of 0.3 was set, it resulted in an F1 score of about 0.79, a recall score of about 0.83, and an average score of about 0.88.

Final Model Selection

The XGBoost model was chosen as the final model. Although some models, such as KNN Model C and Random Forest, had outcomes comparable to or higher F1 scores in specific cases, the XGBoost still provided the best overall combination. It provided high recall (~0.83), which ensured most high-risk cases of displacement are identified, strong precision (~0.75),

which reduced the rate of false positives, high average precision (~0.88), which suggests strong ranking ability, and robust performance across the cross-validation and test data.

Model Performance Assessment

The final XGBoost model demonstrates strong performance in predicting the displacement risk. The relatively high recall shown by the model is important in this context. Failing to identify the at-risk neighborhoods which have much greater and significant consequences than incorrectly flagging some low-risk areas.

The confusion matrix shows relatively few false negatives, which indicates that the model is effective when it comes to capturing true displacement cases. The average precision score of about 0.88 further indicates that the model performs well in ranking high-risk observations about the low-risk ones. (See Appendix C, Figure C9)

Analysis of the feature importance also showed that household income was the most influential predictor, followed by other variables such as food nutrition, demographic factors, housing age, and code violations. This highlights both the importance of socioeconomic and housing factors in determining displacement risk. The XGBoost model offers a strong, reliable solution that balances predictive and accuracy with the ideal goal of identifying high-risk displacement cases. (See Appendix C, Figure C10)

Conclusion

This analysis provides clear evidence that housing unaffordability and displacement risk in Charlotte are driven by a combination of socioeconomic, demographic, and housing-related factors. Through a structured modeling approach, both regression and classification techniques were used to better understand and predict these outcomes. The progression from a basic multiple linear model to a more flexible Generalized Additive Model (GAM) demonstrated the importance of capturing non-linear relationships when modeling housing cost burden. The final GAM, using four key predictors, achieved strong and consistent performance, indicating that a relatively small set of variables can effectively explain much of the variation in housing burden.

Similarly, the classification analysis identified the XGBoost model as the most effective tool for predicting displacement risk. Its ability to maintain high recall ensures that most at-risk neighborhoods are correctly identified, which is especially important in real-world decision-making contexts where missing vulnerable populations can have serious consequences.

Equity Analysis : Historical Redlining (HOLC)

To evaluate whether historical housing discrimination still relates to present-day outcomes in housing prices and displacement, we analyzed neighborhoods in Charlotte that were graded by the Home Owners' Loan Corporation (HOLC) between 1935 and 1968. We compared both observed outcomes (housing cost burden and displacement risk) and model predictions, from the selected GAM and XGBoost models, across HOLC grades (A-D). D-grade neighborhoods were classified as being “hazardous” for investors. Importantly, HOLC grade was not included as a feature in both models. This allowed for independent analysis and comparison.

Present-day neighborhoods with historically graded C and D showed significantly higher housing cost burden and displacement risk than those graded A and B. This pattern appears consistent with the actual data and model predictions. Statistical testing using Kruskal-Wallis test confirms that these differences are significant, in terms of the p-value being less than 0.05, which indicates that the differences in HOLC grades are unlikely to be a result of random variation. (See Appendix D, Table D1) (See Appendix D, Figure D1)

Because the models reproduce the patterns without using HOLC grade, the results suggest that present-day socioeconomic features, like income, home ownership, and food assistance, are structured in ways that reflect historical redlining patterns. While this analysis demonstrates a strong association between past grading and current outcomes, it does not automatically mean causation and is limited by sample size and the use of cross-sectional data.

Beyond predictive performance, this project highlights a deeper structural issue: the lasting impact of historical housing discrimination. The observed relationship between past redlining practices and present-day housing outcomes suggests that current disparities are not random. They are rooted in long-standing systemic inequities. Even without explicitly including historical grading as a feature, both models reflected these patterns, reinforcing the idea that historical disadvantage continues to shape modern socioeconomic conditions.

Overall, this project demonstrates how data-driven modeling can be used not only to predict outcomes but also to uncover meaningful insights about societal challenges. While the models perform well, limitations such as sample size, cross-sectional data, and the presence of outliers suggest that future work could benefit from incorporating longitudinal data and additional contextual variables. Nonetheless, the findings provide a strong foundation for

understanding housing instability and can support policymakers, urban planners, and community organizations in making more informed, equitable decisions.

Appendix A – Data Preparation

Table A1: Master Dataset – Target Variable Distributions

```
1 # Engineer the housing cost burden target variable
2
3 # Calculate raw burden components
4 master_df["rental_cost_burden"] = (master_df["rental_cost"] * 12) / master_df["household_income"]
5 master_df["ownership_cost_burden"] = master_df["home_sales_price"] / (master_df["household_income"] * 3)
6
7 # Normalize both to 0-1 scale before combining
8 scaler = MinMaxScaler()
9 master_df[["rental_cost_burden_scaled", "ownership_cost_burden_scaled"]] = scaler.fit_transform(
10     master_df[["rental_cost_burden", "ownership_cost_burden"]]
11 )
12
13 # Combine using homeownership rate as weight
14 master_df["housing_cost_burden"] = (
15     (master_df["rental_cost_burden_scaled"] * (1 - master_df["home_ownership"] / 100)) +
16     (master_df["ownership_cost_burden_scaled"] * (master_df["home_ownership"] / 100))
17 )
18
19 # Drop intermediate columns
20 master_df = master_df.drop(columns=["rental_cost_burden_scaled", "ownership_cost_burden_scaled"])
21
22 print(master_df["housing_cost_burden"].describe())
```

count	458.000000
mean	0.278200
std	0.165073
min	0.004542
25%	0.150006
50%	0.238862
75%	0.377535
max	0.907929

Name: housing_cost_burden, dtype: float64

Encode Displacement Risk

```
1 # Encode vulnerability to displacement as binary
2 # Yes = 1 (high risk), No = 0 (low risk)
3 master_df["displacement_risk"] = (master_df["vul_class"] == "Yes").astype(int)
4
5 # Verify
6 print(master_df["displacement_risk"].value_counts())
7 print(f"\nHigh risk %: {master_df['displacement_risk'].mean():.2%}")
```

displacement_risk	
0	342
1	116

Name: count, dtype: int64

High risk %: 25.33%

Figure A1: Feature Correlation Heatmap

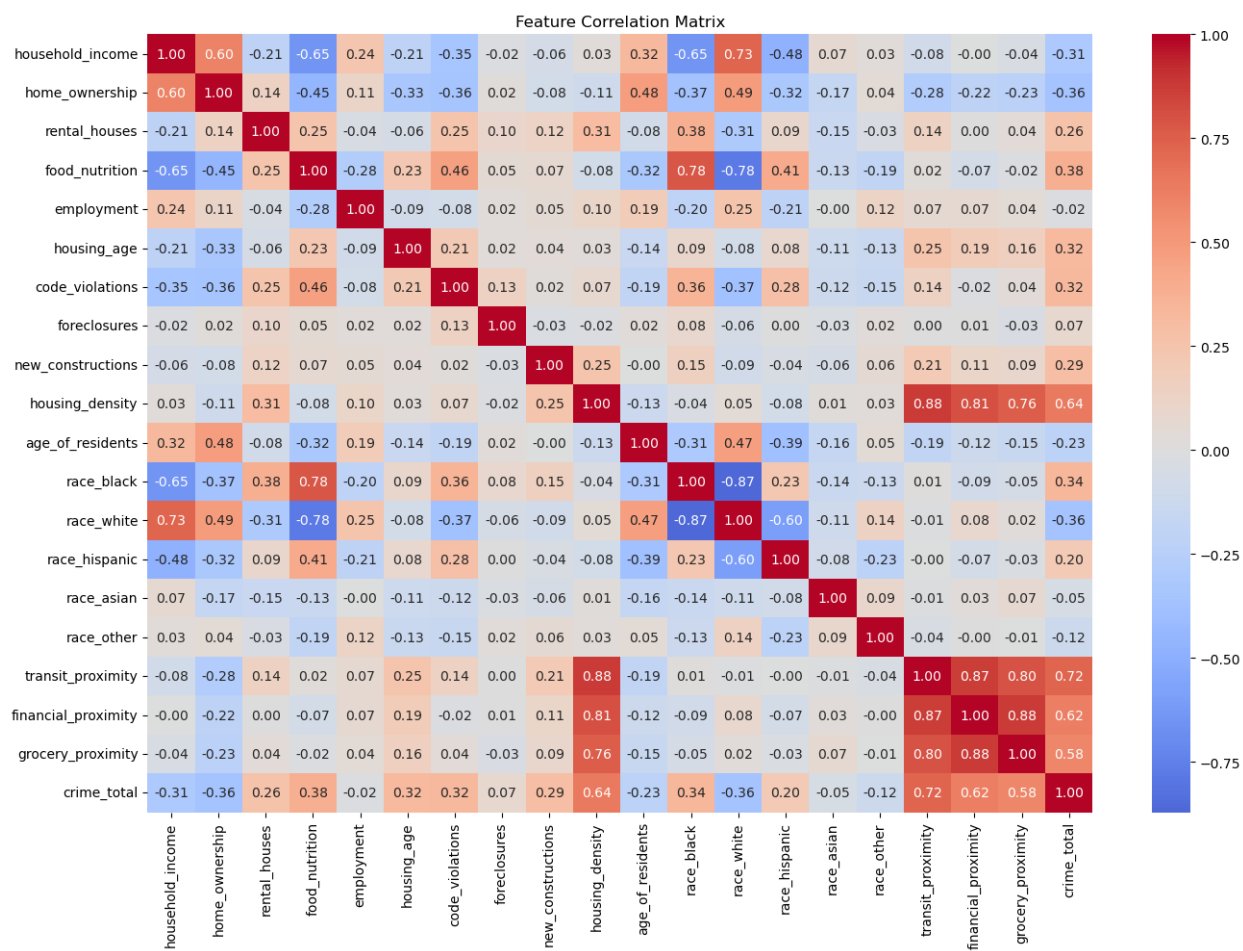


Table A2: Variance Inflation Factor (VIF)

```
1 feature_cols_reg = [c for c in mlr_subset.columns
2 | | | | | if c not in ["NPA", "housing_cost_burden", "displacement_risk"]]
3
4 vif_data = pd.DataFrame()
5 vif_data["feature"] = feature_cols_reg
6 vif_data["VIF"] = [variance_inflation_factor(mlr_subset[feature_cols_reg].values, i)
7 | | | | | for i in range(len(feature_cols_reg))]
8
9 print(vif_data.sort_values("VIF", ascending=False))
```

	feature	VIF
13	transit_proximity	4.581842
9	race_black	3.896286
3	food_nutrition	3.728024
15	crime_total	3.413376
0	household_income	3.131826
14	grocery_proximity	3.028383
1	home_ownership	2.814608
10	race_hispanic	1.884737
8	age_of_residents	1.642831
2	rental_houses	1.583378
6	code_violations	1.492612
5	housing_age	1.318371
11	race_asian	1.224402
7	new_constructions	1.172534
12	race_other	1.164837
4	employment	1.151915

According to the Profiling Report:

- `foreclosures` is imbalanced. 63% of NPAs have 0 foreclosures and has little to no correlation with any of the target variables. This will be dropped for the MLR model.
- `race_white`, `financial_proximity`, and `housing_density` will be dropped for MLR model to lower VIF scores.

Appendix B – Regression Model Output

Table B1: OLS Model Performance (Train/Test)

```
1 # Training metrics
2 y_pred_train = ols_model.predict(X2_train)
3
4 r2_train      = r2_score(y_train, y_pred_train)
5 rmse_train    = root_mean_squared_error(y_train, y_pred_train)
6
7 # Adjusted R2 for train
8 n_train       = len(y_train)
9 k             = X2_train.shape[1] - 1 # exclude constant
10 adj_r2_train = 1 - (1 - r2_train) * (n_train - 1) / (n_train - k - 1)
11
12 # Test metrics
13 r2_test       = r2_score(y_test, y_pred)
14 rmse_test     = root_mean_squared_error(y_test, y_pred)
15
16 # Adjusted R2 for test
17 n_test        = len(y_test)
18 adj_r2_test   = 1 - (1 - r2_test) * (n_test - 1) / (n_test - k - 1)
19
20 # Summary table
21 print("=" * 55)
22 print(f"OLS Model Performance: Housing Cost Burden")
23 print("=" * 55)
24 print(f"{'Metric':<20} {'Train':>15} {'Test':>15}")
25 print("-" * 55)
26 print(f"{'R²':<20} {r2_train:>15.4f} {r2_test:>15.4f}")
27 print(f"{'Adjusted R²':<20} {adj_r2_train:>15.4f} {adj_r2_test:>15.4f}")
28 print(f"{'RMSE':<20} {rmse_train:>15.4f} {rmse_test:>15.4f}")
29 print("=" * 55)
```

OLS Model Performance: Housing Cost Burden

Metric	Train	Test
R ²	0.7356	0.6920
Adjusted R ²	0.7301	0.6719
RMSE	0.0854	0.0892

Table B2: OLS Regression Summary

```
1 #OLS Regression with statsmodels
2
3 # Add constant
4 X2_train = sm.add_constant(X_train)
5
6 # Fit model
7 ols_model = sm.OLS(y_train, X2_train).fit()
8
9 # Predict
10 X2_test = sm.add_constant(X_test)
11 y_pred = ols_model.predict(X2_test)
12
13 # Summary
14 print(ols_model.summary())
15
```

OLS Regression Results						
=====						
Dep. Variable:	housing_cost_burden	R-squared:	0.736			
Model:	OLS	Adj. R-squared:	0.730			
Method:	Least Squares	F-statistic:	133.1			
Date:	Fri, 01 May 2026	Prob (F-statistic):	8.88e-93			
Time:	13:30:44	Log-Likelihood:	357.12			
No. Observations:	343	AIC:	-698.2			
Df Residuals:	335	BIC:	-667.5			
Df Model:	7					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.2766	0.005	59.153	0.000	0.267	0.286
household_income	-0.0507	0.007	-7.205	0.000	-0.065	-0.037
home_ownership	-0.0939	0.007	-14.388	0.000	-0.107	-0.081
food_nutrition	0.0326	0.006	5.505	0.000	0.021	0.044
new_constructions	-0.0096	0.005	-2.010	0.045	-0.019	-0.000
age_of_residents	0.0196	0.005	3.748	0.000	0.009	0.030
transit_proximity	0.0106	0.008	1.258	0.209	-0.006	0.027
grocery_proximity	-0.0103	0.008	-1.281	0.201	-0.026	0.006
=====						
Omnibus:	165.161	Durbin-Watson:	1.937			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1229.734			
Skew:	1.862	Prob(JB):	9.26e-268			
Kurtosis:	11.496	Cond. No.	3.98			
=====						

Figure B1: OLS Residual Diagnostics

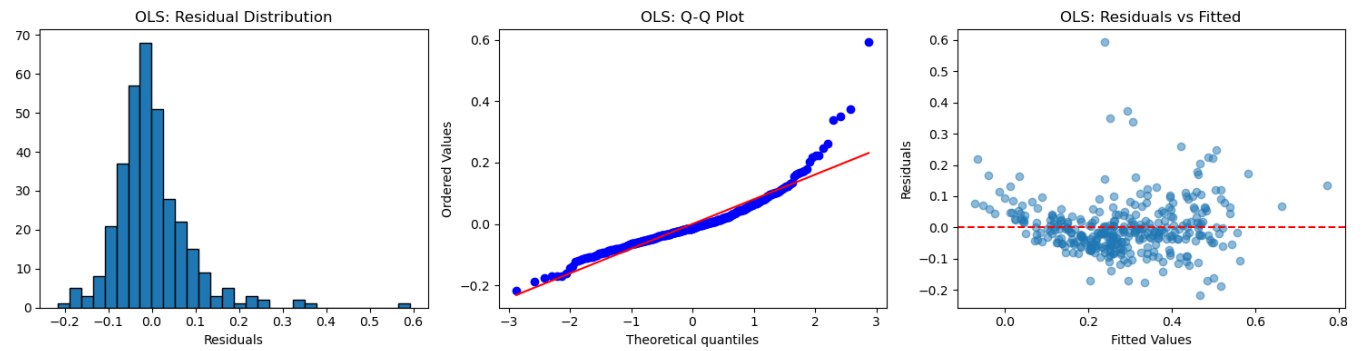


Figure B2: OLS Residuals vs Fitted with LOWESS

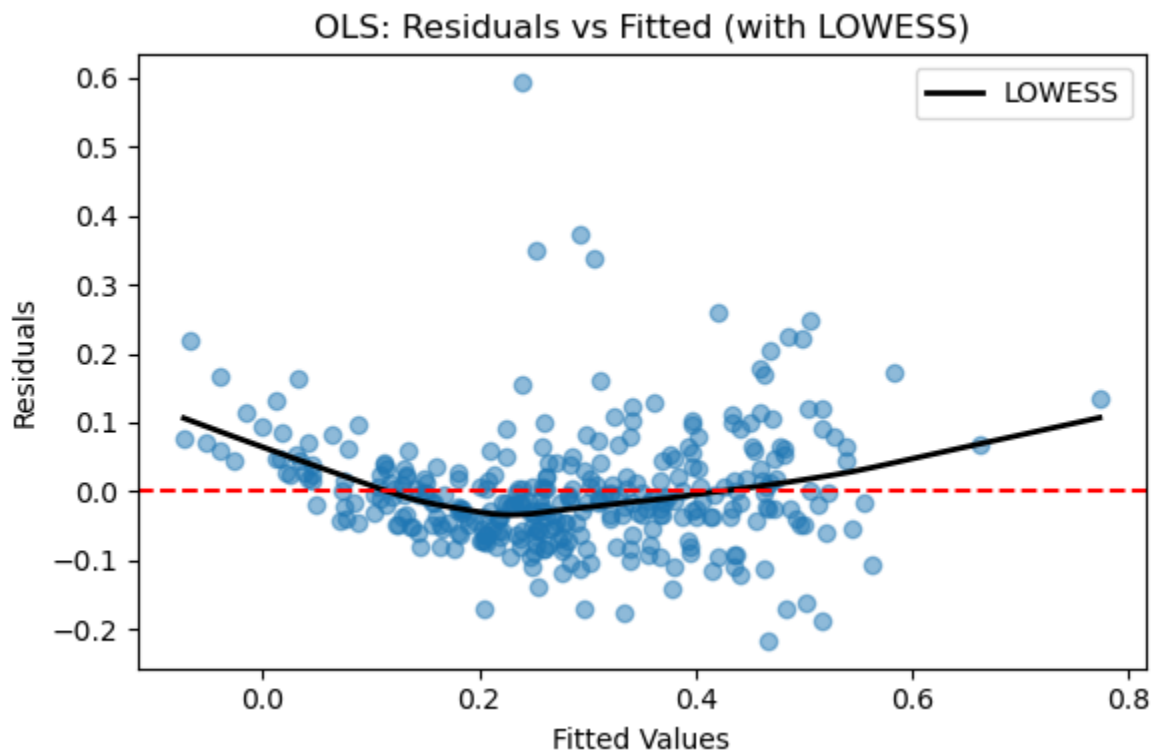


Figure B3: Polynomial Regression Model Comparison

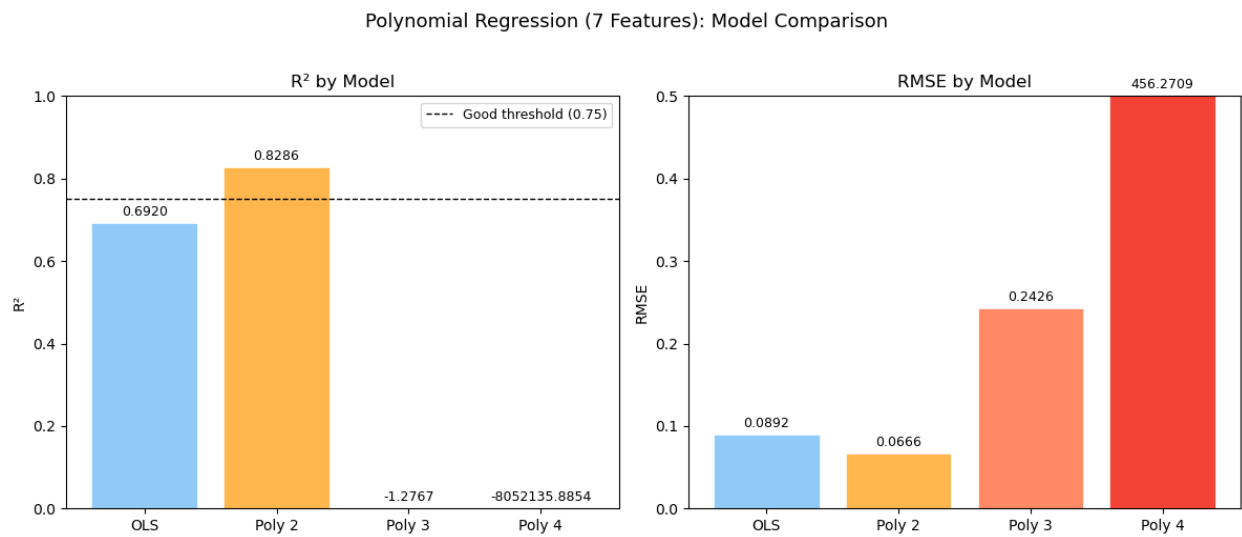


Figure B4: Polynomial Regression: Actual vs Predicted

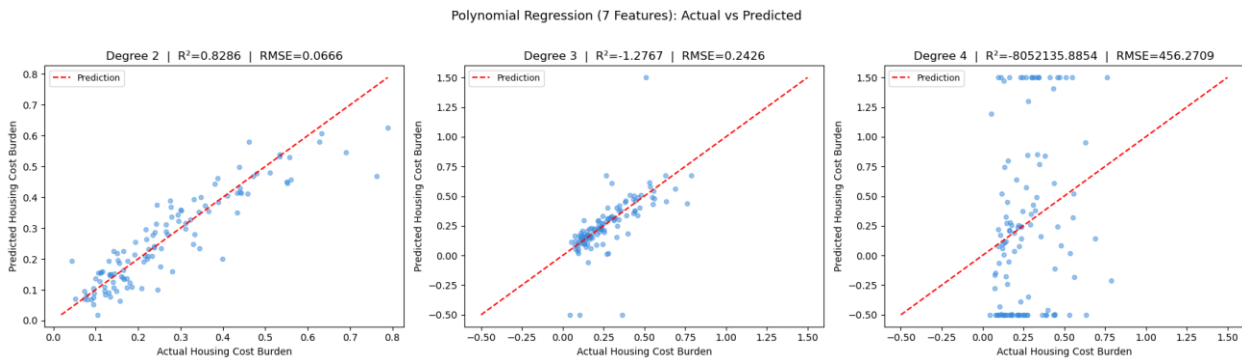


Table B3: GAM Summary: 7 Features

<pre>1 # Fit the GAM 2 # s(i) = spline term for column i (0-indexed) 3 # 0: household_income 1: home_ownership 2: food_nutrition 4 # 3: new_constructions 4: age_of_residents 5: transit_proximity 5 # 6: grocery_proximity 6 7 gam = LinearGAM(s(0) + s(1) + s(2) + s(3) + s(4) + s(5) + s(6)) 8 gam.gridsearch(X_train.values, y_train.values) # auto-tunes lambda (smoothing penalty) 9 10 # Evaluate on test set 11 y_pred_gam = gam.predict(X_test.values) 12 13 r2_gam = r2_score(y_test, y_pred_gam) 14 rmse_gam = root_mean_squared_error(y_test, y_pred_gam) 15 16 print("=" * 40) 17 print("GAM vs OLS Performance on Test Set:") 18 print(f"GAM ... R²: {r2_gam:.4f} RMSE: {rmse_gam:.4f}") 19 print(f"OLS ... R²: {r2_score(y_test, y_pred):.4f} RMSE: {root_mean_squared_error(y_test, y_pred):.4f}") 20 print("=" * 40) 21 print(gam.summary())</pre>						
=====						
GAM vs OLS Performance on Test Set:						
GAM - R²: 0.8273 RMSE: 0.0668						
OLS - R²: -8052135.8854 RMSE: 456.2709						
=====						
LinearGAM						
=====						
Distribution: NormalDist Effective DoF: 21.4994						
Link Function: IdentityLink Log Likelihood: 434.2459						
Number of Samples: 343 AIC: -823.4929						
AICc: -820.1833						
GCV: 0.0056						
Scale: 0.0704						
Pseudo R-Squared: 0.8317						
=====						
Feature Function	Lambda	Rank	EDoF	P > x	Sig. Code	
=====						
s(0)	[63.0957]	20	6.1	1.11e-16	***	
s(1)	[63.0957]	20	4.7	1.11e-16	***	
s(2)	[63.0957]	20	2.2	1.56e-02	*	
s(3)	[63.0957]	20	3.0	9.96e-01		
s(4)	[63.0957]	20	2.6	8.73e-06	***	
s(5)	[63.0957]	20	2.0	7.19e-01		
s(6)	[63.0957]	20	0.8	7.66e-01		
intercept		1	0.0	1.11e-16	***	

Figure B5: GAM Residual Diagnostics: 7 Features

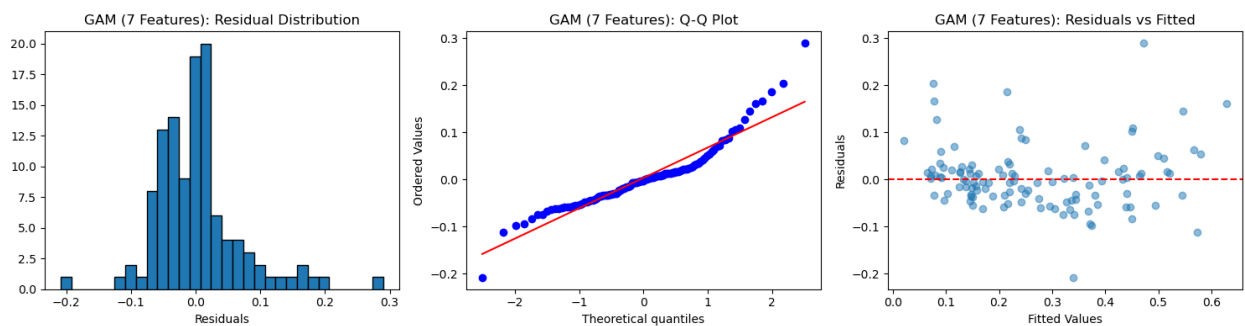


Table B4: GAM Summary: Final 4-Feature Model

```
1 # Fit the GAM
2 # s(i) = spline term for column i (0-indexed)
3 # 0: household_income 1: home_ownership 2: food_nutrition
4 # 3: age_of_residents
5
6 gam = LinearGAM(s(0) + s(1) + s(2) + s(3))
7 gam.gridsearch(X_train.values, y_train.values) # auto-tunes lambda (smoothing penalty)
8
9 # Evaluate on test set
10 y_pred_gam = gam.predict(X_test.values)
11
12 r2_gam = r2_score(y_test, y_pred_gam)
13 rmse_gam = root_mean_squared_error(y_test, y_pred_gam)
14
15 print("=" * 40)
16 print("GAM vs OLS Performance on Test Set:")
17 print(f"GAM   ... R²: {r2_gam:.4f} | RMSE: {rmse_gam:.4f}")
18 print(f"OLS   ... R²: {r2_score(y_test, y_pred):.4f} | RMSE: {root_mean_squared_error(y_test, y_pred):.4f}")
19 print("=" * 40)
20 print(gam.summary())
```

Python

```
=====
GAM vs OLS Performance on Test Set:
GAM   - R²: 0.8321 | RMSE: 0.0659
OLS   - R²: -8052135.8854 | RMSE: 456.2709
=====
LinearGAM
=====
Distribution:          NormalDist Effective DoF:          14.9639
Link Function:         IdentityLink Log Likelihood:       431.613
Number of Samples:     343 AIC:          -831.2982
                        AICc:          -829.637
                        GCV:           0.0054
                        Scale:         0.0703
                        Pseudo R-Squared: 0.8289
=====
Feature Function      Lambda      Rank      EDoF      P > x      Sig. Code
=====
s(0)                  [63.0957]    20       5.9      1.11e-16   ***
s(1)                  [63.0957]    20       4.4      1.11e-16   ***
s(2)                  [63.0957]    20       1.9      1.35e-03   **
s(3)                  [63.0957]    20       2.7      1.45e-05   ***
intercept              1           0.0      1.11e-16   ***
=====
Significance codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Figure B6: GAM Partial Dependence Plots: Final 4-Feature Model

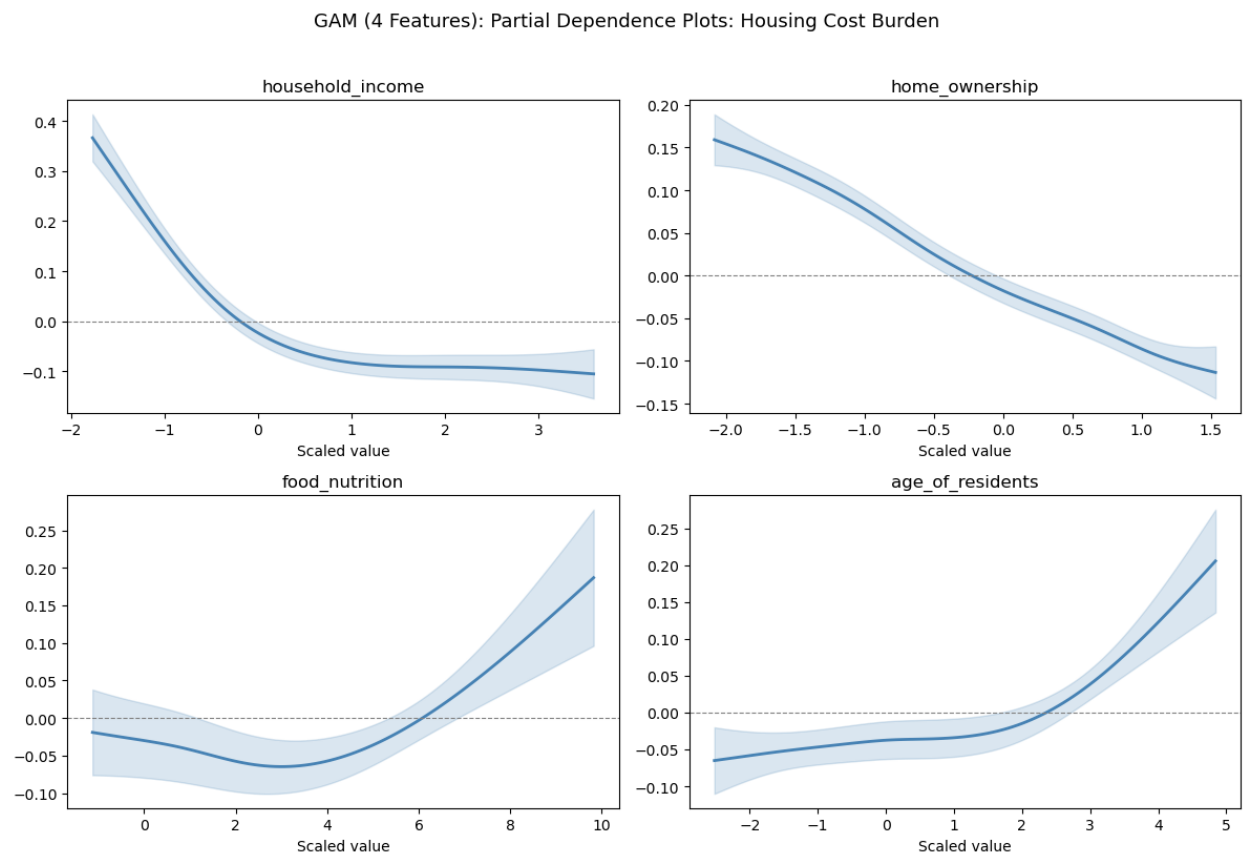


Figure B7: GAM Residual Diagnostics: Final 4-Feature Model

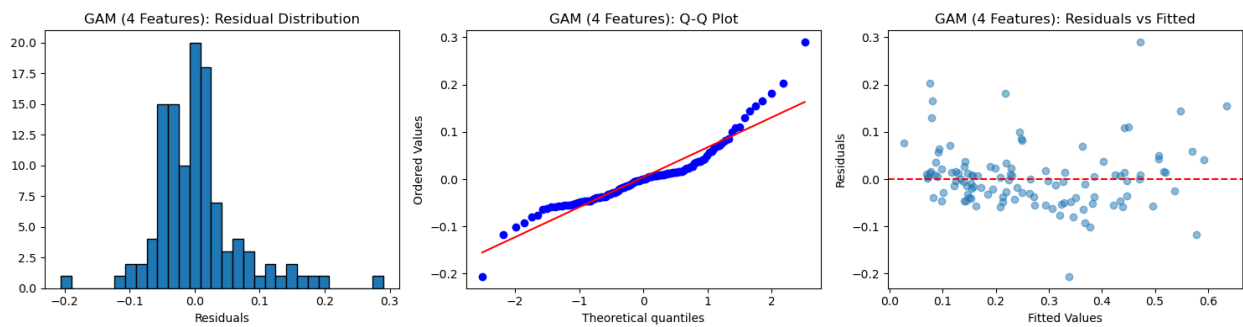


Table B5: GAM 5-Fold Cross-Validation Results

```
1 X_all = pd.concat([X_train, X_test]).values
2 y_all = pd.concat([y_train, y_test]).values
3
4 kf = KFold(n_splits=5, shuffle=True, random_state=42)
5 cv_r2, cv_rmse = [], []
6
7 for fold, (train_idx, val_idx) in enumerate(kf.split(X_all)):
8     X_fold_train, X_fold_val = X_all[train_idx], X_all[val_idx]
9     y_fold_train, y_fold_val = y_all[train_idx], y_all[val_idx]
10
11     gam_fold = LinearGAM(s(0) + s(1) + s(2) + s(3))
12     gam_fold.gridsearch(X_fold_train, y_fold_train)
13
14     y_fold_pred = gam_fold.predict(X_fold_val)
15     fold_r2 = r2_score(y_fold_val, y_fold_pred)
16     fold_rmse = root_mean_squared_error(y_fold_val, y_fold_pred)
17
18     cv_r2.append(fold_r2)
19     cv_rmse.append(fold_rmse)
20     print(f"Fold {fold+1}: R²={fold_r2:.4f} | RMSE={fold_rmse:.4f}")
21
22 print(f"\nMean R²: {np.mean(cv_r2):.4f} ± {np.std(cv_r2):.4f}")
23 print(f"Mean RMSE: {np.mean(cv_rmse):.4f} ± {np.std(cv_rmse):.4f}")
```

Mean R²: 0.8059 ± 0.0815
Mean RMSE: 0.0713 ± 0.0181

Appendix C – Classification Model Output

Figure C1: Logistic Regression Model 1 – Confusion Matrix (All Features, Threshold 0.3)

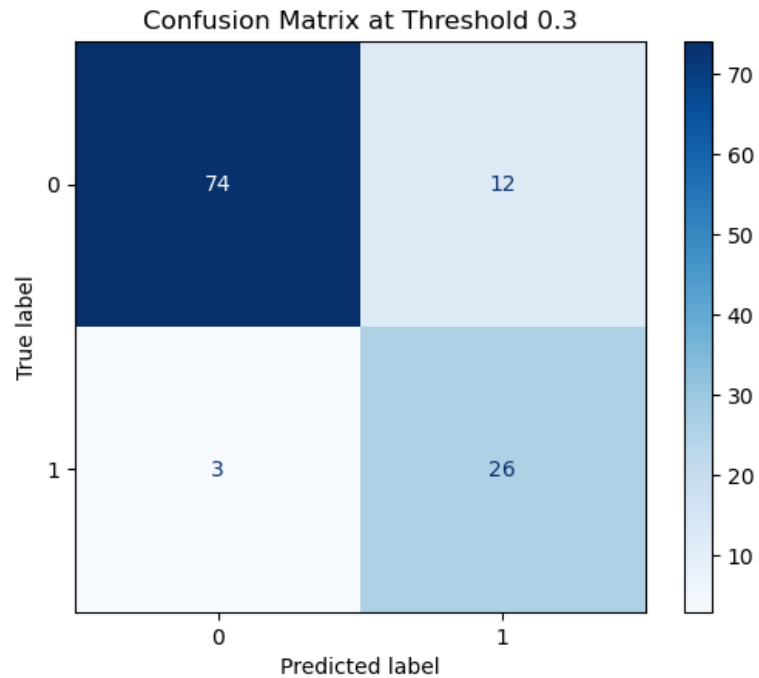


Figure C2: Logistic Regression Model 2 — Confusion Matrix (5 Manually Selected Features)

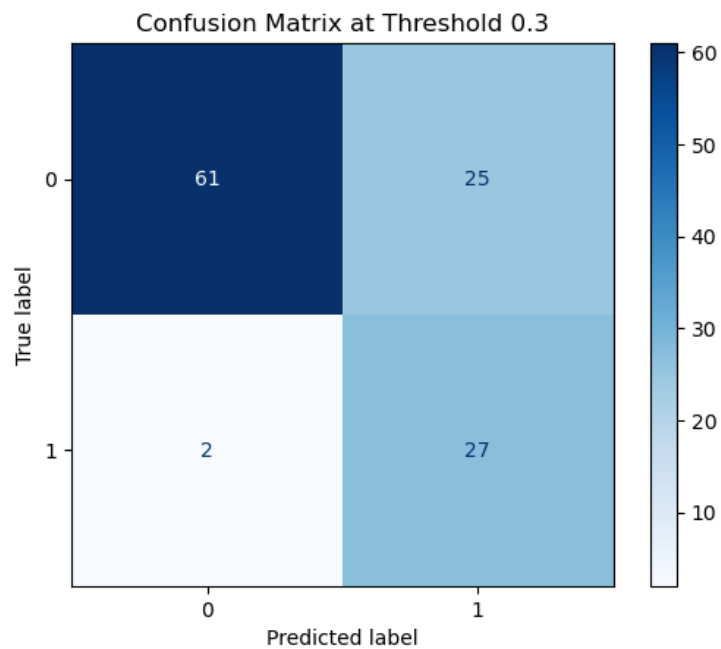


Figure C3: Logistic Regression Model 3 — Confusion Matrix (Informed Feature Selection)

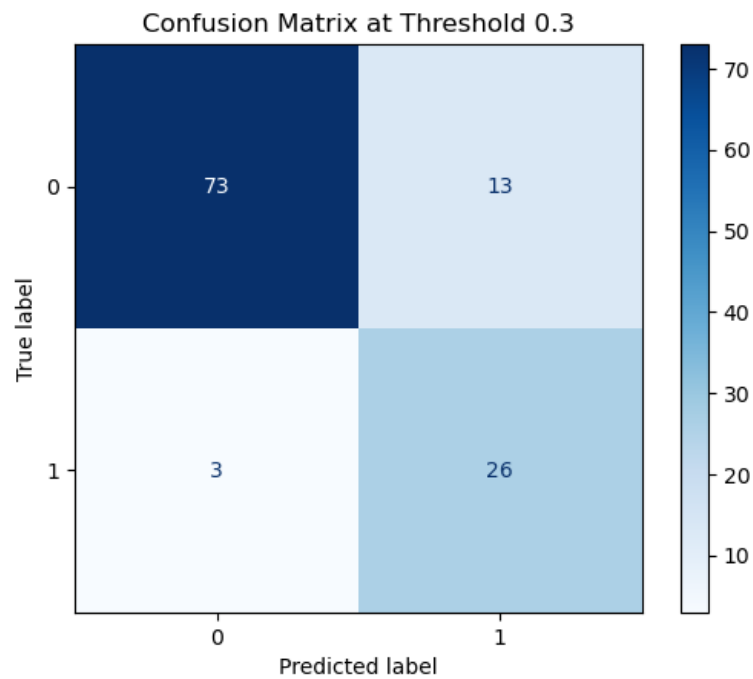


Figure C4: Random Forest — Confusion Matrix and Feature Importance

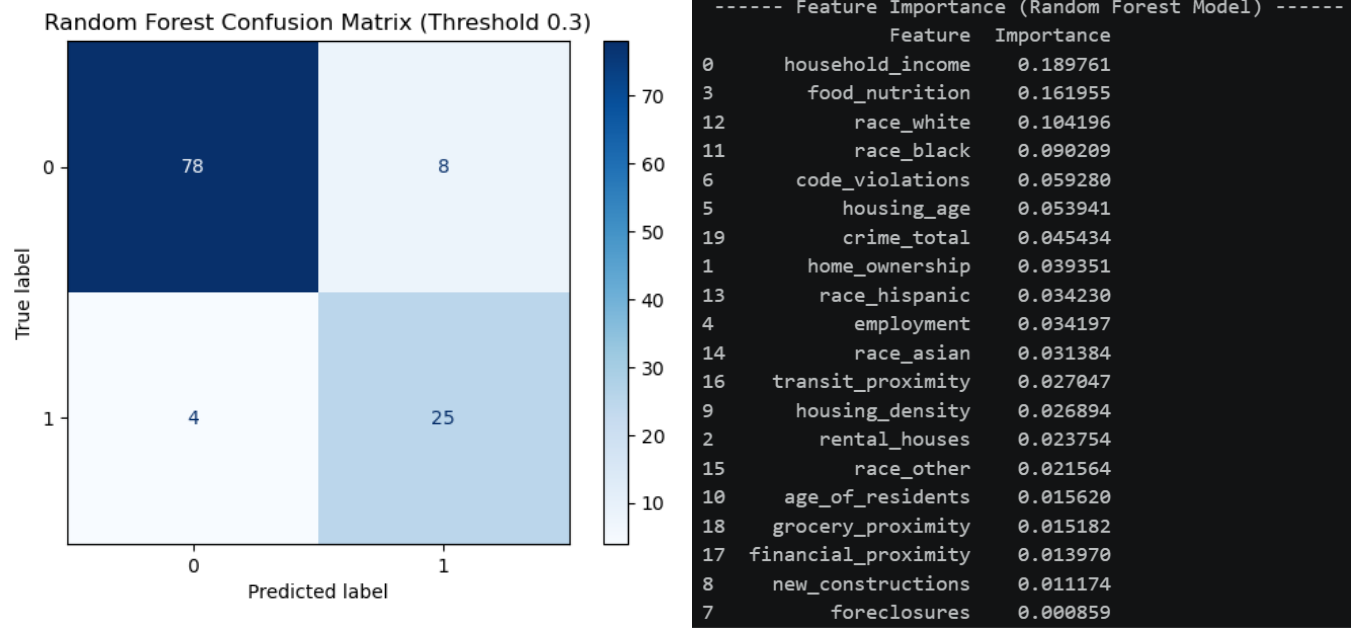


Figure C5: KNN Model A — Confusion Matrix (All Features, k=3)

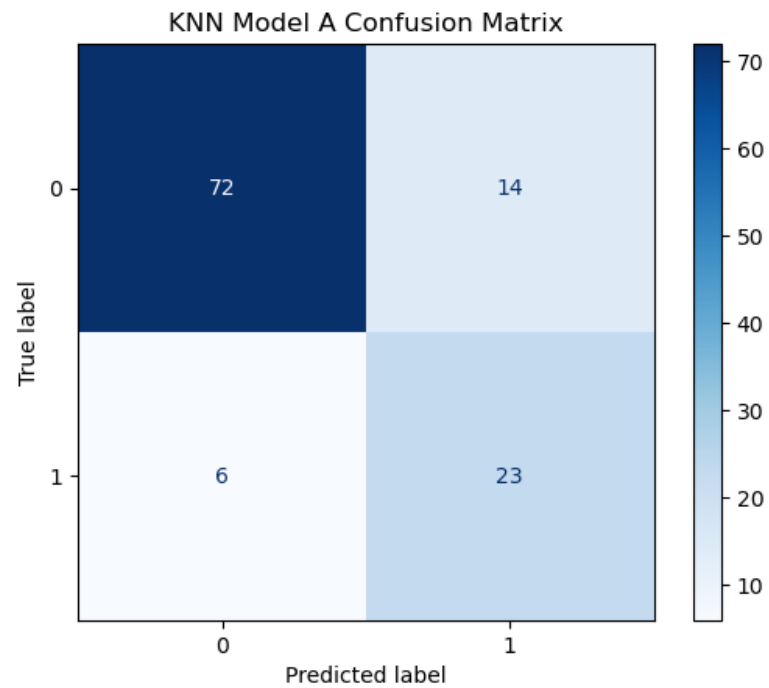


Figure C6: KNN Model B — Confusion Matrix (Top 5 Logistic Regression Features, k=9)

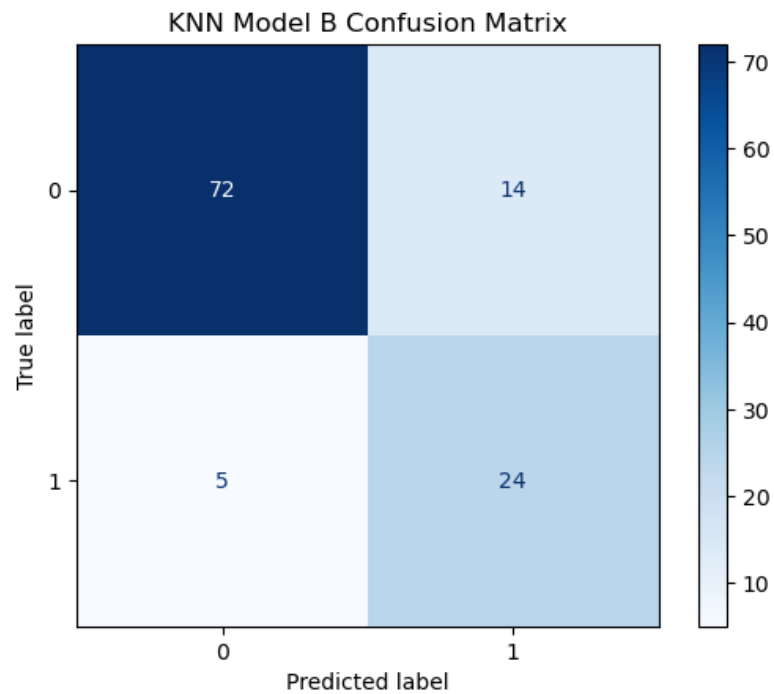


Figure C7: KNN Model C — Confusion Matrix (Top 5 Random Forest Features, k=9)

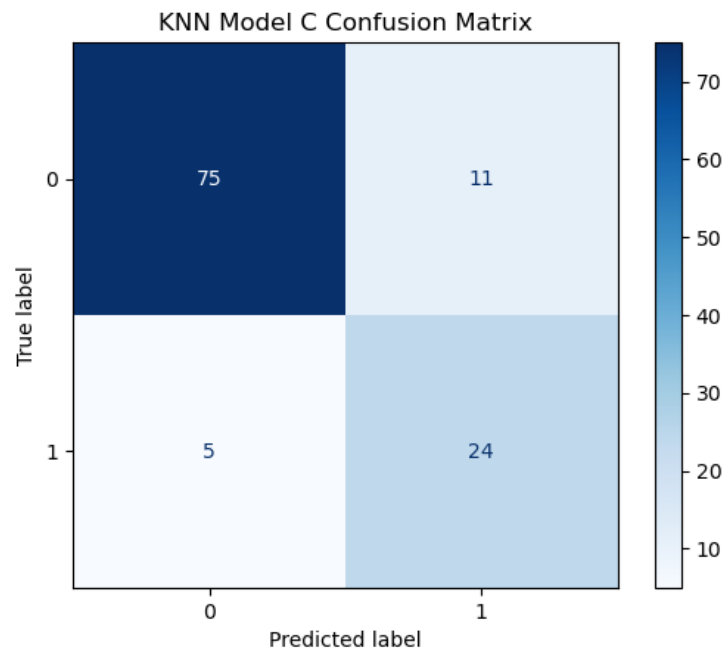


Figure C8: KNN Model D — Confusion Matrix (Top 10 Intersection Features, k=3)

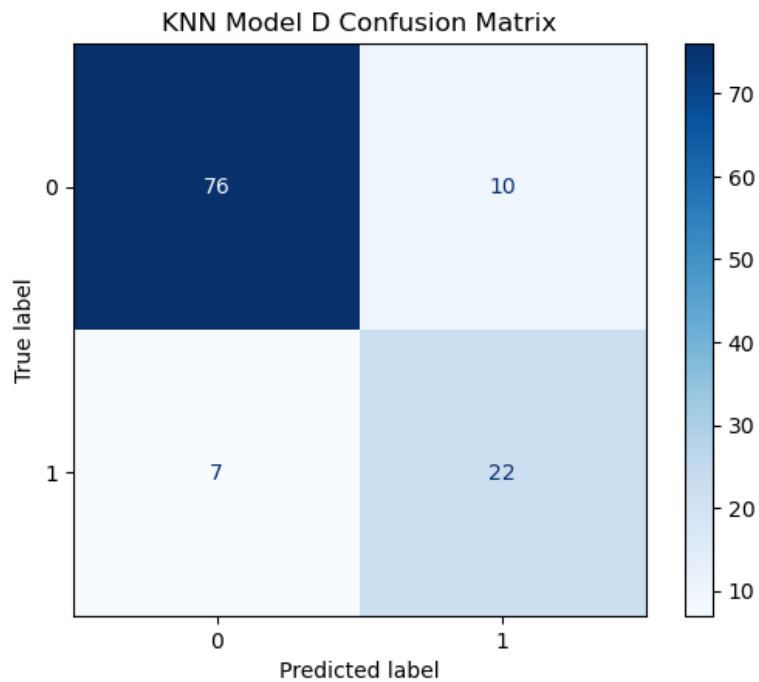


Figure C9: XGBoost — Confusion Matrix (Final Model, Threshold 0.3)

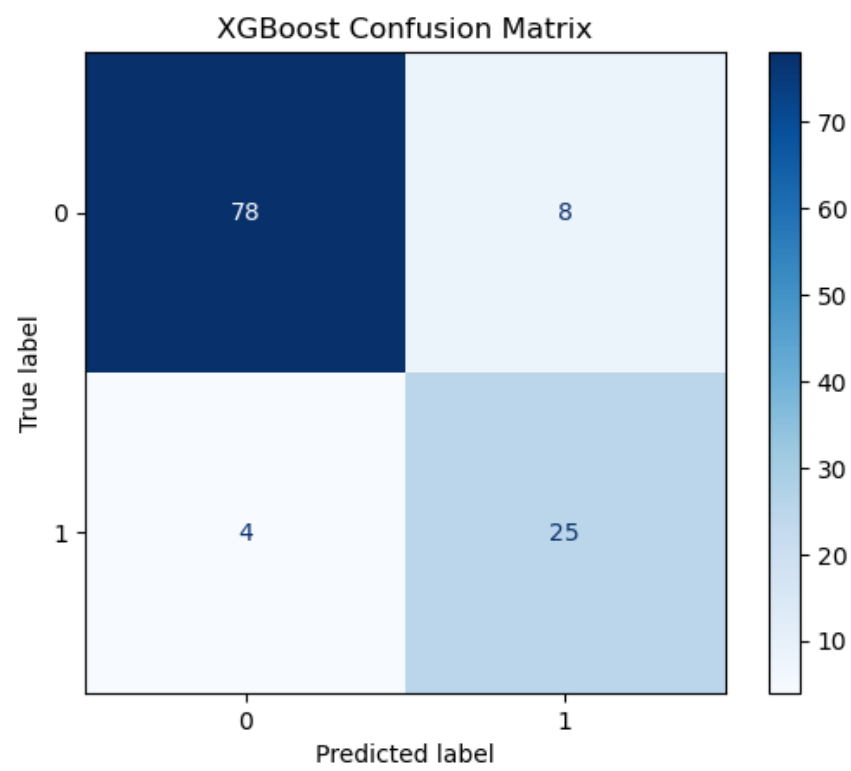


Figure C10: XGBoost — Feature Importance

```
--- Feature Importance (XGBoost Model) ---
```

	Feature	Importance
0	household_income	0.259954
3	food_nutrition	0.114464
12	race_white	0.066727
11	race_black	0.060005
13	race_hispanic	0.056171
5	housing_age	0.049388
6	code_violations	0.048770
4	employment	0.035231
8	new_constructions	0.034385
14	race_asian	0.032676
1	home_ownership	0.032565
9	housing_density	0.032223
15	race_other	0.031992
2	rental_houses	0.027848
10	age_of_residents	0.026825
18	grocery_proximity	0.023544
19	crime_total	0.023225
16	transit_proximity	0.022152
17	financial_proximity	0.021855
7	foreclosures	0.000000

Appendix D – HOLC Post-Analysis Output

Figure D1: HOLC Grade Analysis: Four Panel Visualization

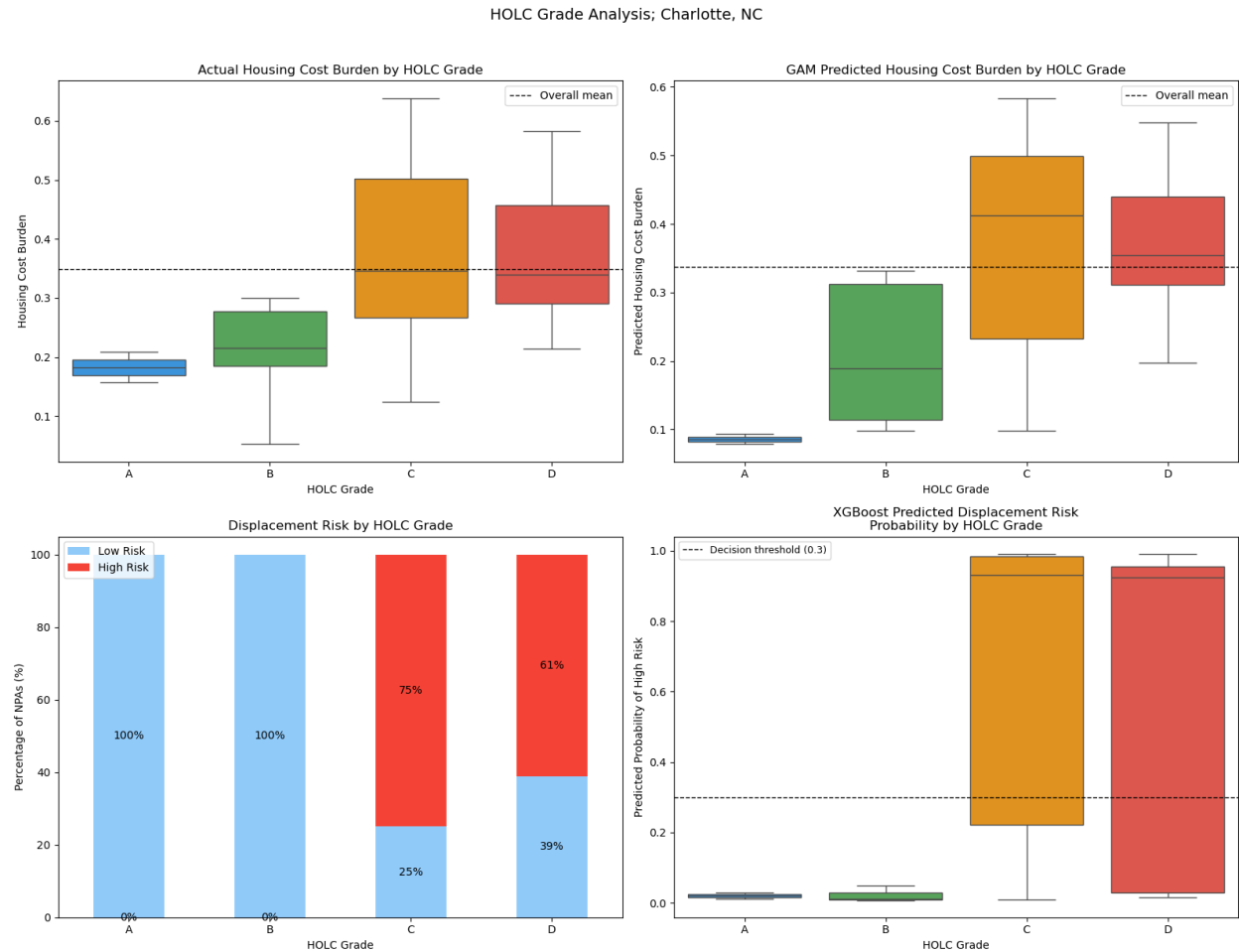


Table D1: Kruskal-Wallis Test Results for Actual Outcomes and Predicted Outcomes

```
1 # Kruskal-Wallis on housing_cost_burden
2 groups_burden = [
3     holc_df[holc_df["grade"] == g]["housing_cost_burden"].values
4     for g in ["A", "B", "C", "D"]
5 ]
6
7 h_burden, p_burden = kruskal(*groups_burden)
8 print("=" * 55)
9 print("Kruskal-Wallis Test: Housing Cost Burden")
10 print("=" * 55)
11 print(f"H statistic: {h_burden:.4f}")
12 print(f"P-value:      {p_burden:.4f}")
13 print(f"Result:      {'Significant' if p_burden < 0.05 else 'Not significant'}")
14
15 # Kruskal-Wallis on displacement_risk
16 groups_risk = [
17     holc_df[holc_df["grade"] == g]["displacement_risk"].values
18     for g in ["A", "B", "C", "D"]
19 ]
20
21 h_risk, p_risk = kruskal(*groups_risk)
22 print()
23 print("=" * 55)
24 print("Kruskal-Wallis Test: Displacement Risk")
25 print("=" * 55)
26 print(f"H statistic: {h_risk:.4f}")
27 print(f"P-value:      {p_risk:.4f}")
28 print(f"Result:      {'Significant' if p_risk < 0.05 else 'Not significant'})
```

```
=====
Kruskal-Wallis Test: Housing Cost Burden
=====
H statistic: 9.6994
P-value:      0.0213
Result:      Significant

=====
Kruskal-Wallis Test: Displacement Risk
=====
H statistic: 11.1702
P-value:      0.0108
Result:      Significant
```

```

1 # Kruskal-Wallis on GAM predicted burden
2 groups_gam = [
3     holc_df[holc_df["grade"] == g]["gam_predicted_burden"].values
4     for g in ["A", "B", "C", "D"]
5 ]
6
7 h_gam, p_gam = kruskal(*groups_gam)
8 print("=" * 55)
9 print("Kruskal-Wallis Test: GAM Predicted Cost Burden")
10 print("=" * 55)
11 print(f"H statistic: {h_gam:.4f}")
12 print(f"P-value:      {p_gam:.4f}")
13 print(f"Result:      {'Significant' if p_gam < 0.05 else 'Not significant'}")
14
15 # Kruskal-Wallis on XGBoost predicted probability
16 groups_xgb = [
17     holc_df[holc_df["grade"] == g]["xgb_proba"].values
18     for g in ["A", "B", "C", "D"]
19 ]
20
21 h_xgb, p_xgb = kruskal(*groups_xgb)
22 print()
23 print("=" * 55)
24 print("Kruskal-Wallis Test: XGBoost Predicted Risk Probability")
25 print("=" * 55)
26 print(f"H statistic: {h_xgb:.4f}")
27 print(f"P-value:      {p_xgb:.4f}")
28 print(f"Result:      {'Significant' if p_xgb < 0.05 else 'Not significant'}")

```

```

=====
Kruskal-Wallis Test: GAM Predicted Cost Burden
=====

```

```
H statistic: 10.8079
```

```
P-value:      0.0128
```

```
Result:      Significant
```

```

=====
Kruskal-Wallis Test: XGBoost Predicted Risk Probability
=====

```

```
H statistic: 11.0052
```

```
P-value:      0.0117
```

```
Result:      Significant
```

Appendix E — Database and GitHub Evidence

Figure E1: SQLite Database Schema

```
=== clt_holc ===
area_id (INTEGER)
city (TEXT)
state (TEXT)
category (TEXT)
grade (TEXT)
residential (INTEGER)
commercial (INTEGER)
industrial (INTEGER)

~
=== npa ===
npa (INTEGER)
land_area (INTEGER)
transit_access_score (REAL)
jobs_accessible_transit (REAL)
jobs_accessible_car (REAL)
neighborhood_change_score (REAL)
median_household_income (INTEGER)
permits_per_1000 (REAL)
median_sales_price (INTEGER)
sales_price_change (REAL)
food_nutrition_services_2016 (REAL)
poverty_rate_median (INTEGER)

~
=== npa_holc ===
npa (INTEGER)
grade (TEXT)
grade_category (TEXT)

~
=== qol ===
NPA (INTEGER)
household_income (REAL)
home_ownership (REAL)
home_sales_price (REAL)
rental_cost (REAL)
rental_houses (REAL)
food_nutrition (REAL)
health_insurance (REAL)
employment (REAL)
housing_age (REAL)
code_violations (REAL)
foreclosures (REAL)
new_constructions (REAL)
housing_density (REAL)
age_of_residents (REAL)
race_black (REAL)
race_white (REAL)
race_hispanic (REAL)
race_asian (REAL)
race_other (REAL)
transit_proximity (REAL)
financial_proximity (REAL)
grocery_proximity (REAL)
crime_property (REAL)
crime_violent (REAL)

~
=== vul_disp ===
OBJECTID (INTEGER)
ID (INTEGER)
GlobalID (TEXT)
NPA (INTEGER)
PctHighSchool (TEXT)
PctNonWhite (TEXT)
Pct65_ (TEXT)
PctHH_Poverty (TEXT)
PctHome_Ownership (TEXT)
HighSchool (REAL)
NonWhite (REAL)
F65_ (REAL)
Poverty (REAL)
Home_Ownership (REAL)
Score (REAL)
Vulnerable (TEXT)
ShapeSTArea (REAL)
ShapeSTLength (REAL)

~
=== master ===
NPA (INTEGER)
household_income (REAL)
home_ownership (REAL)
rental_houses (REAL)
food_nutrition (REAL)
employment (REAL)
housing_age (REAL)
code_violations (REAL)
foreclosures (REAL)
new_constructions (REAL)
housing_density (REAL)
age_of_residents (REAL)
race_black (REAL)
race_white (REAL)
race_hispanic (REAL)
race_asian (REAL)
race_other (REAL)
transit_proximity (REAL)
financial_proximity (REAL)
grocery_proximity (REAL)
housing_cost_burden (REAL)
displacement_risk (INTEGER)
crime_total (REAL)

~
=== gam_predictions ===
NPA (INTEGER)
gam_predicted_burden (REAL)

~
=== xgb_predictions ===
NPA (INTEGER)
xgb_proba (REAL)
xgb_pred (INTEGER)
```

Figure E2: GitHub Repository

<https://github.com/coffeeNqueries/DTSC2302-Project3>